

Chapter 1: Project Overview

1.1 Introduction

Construction Project Management & Accounting SaaS is a comprehensive software platform designed to help construction companies, contractors, and project-based businesses manage their complete project operations, financial activities, workforce management, and business reporting from a single centralized system.

The construction industry follows a project-based business model where every project has its own client, contract, budget, timeline, workforce requirements, subcontractors, expenses, and payment cycles. Managing these activities efficiently becomes challenging when a company handles multiple projects simultaneously.

Many small and medium-sized construction businesses still depend on spreadsheets, manual registers, and disconnected tools to maintain their project and financial information. These traditional processes often create difficulties in tracking project costs, calculating employee wages, managing subcontractor payments, monitoring client payments, and determining actual project profitability.

The purpose of this system is to provide a structured digital platform where construction businesses can manage their complete project lifecycle, financial transactions, workforce, documents, and reports efficiently.

This platform will not only function as an accounting solution but also as a complete construction management system that combines project management, party management, employee management, payment tracking, expense management, document management, and reporting capabilities.

1.2 Project Objectives

The primary objective of this project is to simplify and automate construction business management processes through a scalable and flexible software solution.

The system will enable construction owners and contractors to manage multiple projects, maintain business relationships, track financial activities, manage employees, control expenses, and generate meaningful reports.

The key objectives of this project are:

- Provide a centralized platform for managing construction projects
- Maintain complete project information and history
- Manage client, owner, supplier, and subcontractor relationships
- Track project income, expenses, and profitability
- Support employee attendance and wage calculation

- Manage daily wage and monthly salary systems
- Track subcontractor contracts and payments
- Manage company-owned machinery and rented equipment
- Store project documents, designs, agreements, and site photos
- Generate project and financial reports for better decision-making

The system will reduce manual work, improve accuracy, and provide better visibility into construction business operations.

1.3 Product Vision

The long-term vision of Construction Project Management & Accounting SaaS is to build a scalable Construction ERP platform that can support small contractors, medium construction companies, and growing construction organizations.

The initial version of the system will focus on essential project management and accounting functionalities required by construction businesses. The architecture will be designed with future expansion in mind so that additional modules can be introduced without major changes to the existing system.

The platform will gradually evolve by adding advanced construction management capabilities such as:

- Inventory Management
- Material Management
- Procurement Management
- BOQ Management
- Client Portal
- Site Management Application
- Task Management
- Quality Management
- Safety Management
- Advanced Analytics
- Mobile Application Support

The system architecture will follow a modular approach so that each feature can be developed, maintained, and extended independently.

1.4 Target Users

The system is designed for different types of users involved in construction business operations.

Construction Company Owner

Construction owners will use the system to monitor overall business performance, manage projects, track financial activities, and make informed decisions.

They will be able to view:

- Active projects
- Project financial status
- Client payment status
- Total expenses
- Employee costs
- Subcontractor payments
- Project profitability

Contractors

Contractors will use the platform to manage client projects, employees, subcontractors, expenses, and payments from a single system.

Project Managers

Project managers will manage project activities, site updates, employee assignments, progress information, and project documentation.

Accountants

Accountants will manage financial transactions, expenses, payments, party ledgers, and financial reports.

Site Supervisors

Site supervisors will update site-related information, employee attendance, project progress, and upload site photos or documents.

1.5 Problem Statement

Construction businesses face several challenges due to manual and disconnected management processes.

Managing multiple projects requires continuous tracking of project costs, payments, workforce activities,

subcontractor expenses, and operational activities. Without a centralized system, it becomes difficult to maintain accurate records and obtain real-time business insights.

Employee management is another major challenge because construction companies often use different payment models, including daily wages, monthly salaries, and contract-based payments. Manual calculation of attendance and wages can lead to errors and payment disputes.

Subcontractor management also requires proper tracking of work assignments, contract values, advance payments, completed work, and outstanding payments.

Additionally, construction projects contain many important documents such as agreements, drawings, designs, approvals, and site photos. Managing these documents separately can create difficulties during project execution and future reference.

This system will address these challenges by providing a centralized, organized, and scalable construction management platform.

1.6 Project Scope

The initial scope of the system will cover the core requirements of construction project management and accounting.

The first version will include the following major areas:

Project Management

The system will allow users to create and manage multiple construction projects.

Each project will contain:

- Project details
- Client/Party information
- Contract information
- Project timeline
- Project status
- Related documents
- Financial activities

Party Management

The system will provide a unified party management system where different business relationships can be managed.

Supported party types will include:

- Client
- Owner
- Supplier
- Subcontractor
- Consultant
- Other business parties

Employee Management

The system will support:

- Employee profiles
- Project assignment
- Attendance tracking
- Daily wage calculation
- Monthly salary calculation
- Payment history

Finance Management

The finance module will manage:

- Income
- Expenses
- Payments
- Party transactions
- Ledger
- Financial reports

Subcontractor Management

The system will support:

- Subcontractor information
- Work assignment
- Contract details
- Payment tracking
- Due management

Machinery and Equipment Management

The system will support:

- Company-owned machinery records
- Maintenance history
- Equipment rental tracking
- Usage information
- Related expenses

Document Management

The system will allow users to store:

- Contracts
- Agreements
- Drawings
- Designs
- Site photos
- Project documents

Reporting System

The system will provide reports related to:

- Project cost
- Expense summary
- Payment history
- Employee cost
- Profit and loss analysis

1.7 Deployment Model

The system will support both SaaS-based deployment and self-hosted deployment models.

SaaS Deployment

The application will operate as a multi-tenant SaaS platform where multiple construction companies can use the same application while maintaining complete data isolation.

Each tenant will have:

- Independent company profile
- Independent users
- Independent projects

- Independent transactions
- Independent configurations

Self-Hosted Deployment

The system will support private deployment where a company can host the application on its own infrastructure.

Self-hosted deployment will support:

- Custom domain
- Own database
- Custom branding
- Independent operation

The same core application architecture should support both deployment models.

1.8 Design Goals

The system will be designed based on the following principles:

Scalability

The architecture should support increasing numbers of users, projects, transactions, and business requirements.

Modularity

Each business module should be independent and loosely coupled to allow future expansion.

Maintainability

The codebase should follow clean architecture principles so that future development and maintenance remain simple.

Flexibility

The system should support different construction business models, contract types, and operational workflows.

AI-Assisted Development Compatibility

The project structure, documentation, business rules, and development process should be optimized for AI-assisted development using tools such as Cursor and other coding assistants.

Future Expansion

The architecture should allow additional modules to be added without affecting existing functionality.

1.9 Success Criteria

The project will be considered successful when construction businesses can efficiently manage their complete operations through a single platform.

The system should enable users to:

- Manage multiple construction projects
- Track project financial performance
- Control expenses
- Manage employee wages and payments
- Monitor subcontractor activities
- Maintain party relationships
- Store project documents
- Generate useful business reports

The ultimate goal is to create a reliable, scalable, and future-ready Construction ERP platform that improves operational efficiency and financial control for construction businesses.

End of Chapter 1: Project Overview

Chapter 2: System Architecture Design

2.1 Introduction

The architecture of Construction Project Management & Accounting SaaS will be designed to support scalability, maintainability, flexibility, and future expansion.

The system will follow a modular architecture approach where each business capability will be developed as an independent module with clear responsibilities and well-defined communication between modules.

The primary goal of this architecture is to create a system that can support both SaaS deployment and self-hosted deployment while allowing new features and business modules to be added easily in the future.

The architecture will also be optimized for AI-assisted development so that each module can be developed, tested, and improved independently with minimum complexity.

2.2 Architecture Principles

The system architecture will follow the following principles:

Modular Design

The application will be divided into multiple independent modules based on business functionality.

Each module will have:

- Defined responsibility
- Independent business logic
- Separate data handling
- Clear communication interface

This approach will reduce dependency between modules and make future development easier.

Scalability

The architecture should support business growth without requiring major structural changes.

The system should be capable of handling:

- Increasing number of companies
 - Increasing number of users
 - Multiple projects per company
 - Large transaction volumes
 - Additional future modules
-

Maintainability

The system should follow clean architecture principles to ensure that future developers can easily understand, modify, and extend the application.

The codebase should maintain:

- Clear folder structure
- Separation of concerns
- Reusable components

- Proper documentation
 - Consistent coding standards
-

Flexibility

Construction businesses may have different workflows, contract types, and operational requirements.

The architecture should provide flexibility to support:

- Different project types
 - Different payment models
 - Different employee salary systems
 - Different business configurations
-

AI-Friendly Development

Since the project will be developed with AI coding assistants, the architecture and documentation should be structured in a way that allows AI tools to understand the system easily.

Each module should contain:

- Clear requirements
- Business rules
- Database definitions
- API specifications
- Validation rules
- Permission rules

This will reduce unnecessary context usage and improve AI-generated code quality.

2.3 High-Level System Architecture

The system will follow a layered architecture approach.

The major layers will include:

Presentation Layer

Responsible for user interaction.

Responsibilities:

- Web application interface
 - Mobile responsive interface
 - Dashboard
 - Forms
 - Reports
 - User experience
-

Application Layer

Responsible for application logic and business workflow.

Responsibilities:

- Processing user requests
 - Executing business rules
 - Managing module communication
 - Handling transactions
-

Domain Layer

Responsible for core business logic.

Examples:

- Project rules
 - Payment rules
 - Payroll calculation
 - Expense rules
 - Party transactions
-

Data Layer

Responsible for data storage and retrieval.

Responsibilities:

- Database operations
 - Data relationships
 - Data integrity
 - Transaction management
-

2.4 Module Architecture

The system will be divided into two major categories:

Core System Modules

Core modules will provide common functionality required by all business modules.

Core modules include:

- Authentication
 - User Management
 - Role and Permission Management
 - Tenant Management
 - Company Settings
 - Notification System
 - Audit Logging
-

Business Modules

Business modules will contain construction-specific functionality.

Business modules include:

- Party Management
- Project Management
- Contract Management
- Employee Management

- Payroll Management
 - Expense Management
 - Finance Management
 - Machinery Management
 - Equipment Rental Management
 - Document Management
 - Reporting System
-

2.5 Core Module Responsibilities

Authentication Module

The authentication module will manage user identity and access control.

Responsibilities:

- User login
 - Password management
 - Session management
 - Authentication security
-

Tenant Management Module

The tenant module will manage company-level separation in the SaaS environment.

Responsibilities:

- Create tenant
 - Manage company information
 - Isolate tenant data
 - Maintain tenant configuration
-

User Management Module

The user management module will manage system users.

Responsibilities:

- User creation
 - User profile management
 - User status
 - User assignment
-

Role and Permission Module

This module will control access to system features.

Responsibilities:

- Role creation
 - Permission assignment
 - Module access control
 - Action-level permission management
-

Notification Module

The notification module will handle system alerts and reminders.

Responsibilities:

- Payment reminders
 - Salary reminders
 - Project deadline alerts
 - Equipment return reminders
 - System notifications
-

Audit Log Module

The audit system will record important user activities.

Examples:

- Created project
- Updated payment
- Deleted transaction
- Modified settings

This will improve security and accountability.

2.6 Business Module Architecture

Each business module will follow a similar internal structure.

Example:

Project Module:

- Project Controller
- Project Service
- Project Business Rules
- Project Repository
- Project Database Models
- Project API
- Project Validation

This structure will be followed for all major modules.

2.7 SaaS Multi-Tenant Architecture

The system will support multi-tenant SaaS architecture.

Each company using the platform will be treated as a separate tenant.

Tenant isolation will ensure:

- One company cannot access another company's data
- Separate business configuration
- Separate users
- Separate transactions

The architecture should support different tenant strategies depending on deployment requirements.

Possible approaches:

- Shared database with tenant isolation
- Separate database per tenant

The final implementation approach will be selected based on scalability, security, and deployment

requirements.

2.8 Self-Hosted Architecture

The same application architecture will support private deployment.

A self-hosted client will have:

- Independent application instance
- Independent database
- Custom domain
- Custom configuration

The application should avoid unnecessary dependency on external SaaS services so that it can operate independently.

2.9 Event-Driven Communication

The system will use event-based communication for important business operations.

Instead of directly updating multiple modules, business events will trigger required actions.

Example:

Employee Salary Payment Created

?

Finance Transaction Created

?

Project Cost Updated

?

Reports Updated

Another example:

Client Payment Received

?

Party Ledger Updated

?

Project Income Updated

?

Financial Reports Updated

This approach will improve scalability and reduce module dependency.

2.10 Data Flow Principle

The system will follow a single source of truth approach.

Important business data should be stored once and reused across modules.

Example:

Party information will be managed through Party Management Module and reused by:

- Project Module
- Finance Module
- Supplier Management
- Subcontractor Management

This approach will prevent duplicate data and maintain consistency.

2.11 Future Module Integration

Future modules should be integrated without changing the existing core architecture.

New modules should follow:

- Independent structure
- Defined responsibilities
- Existing authentication system
- Existing permission system
- Existing notification system
- Existing reporting framework

Examples of future modules:

- Inventory
 - Procurement
 - BOQ
 - Client Portal
 - Mobile Application
-

2.12 Architecture Summary

The Construction Project Management & Accounting SaaS will use a modular, scalable, and future-ready architecture.

The system will separate core functionality from business modules, support SaaS and self-hosted deployment, and allow future expansion without major architectural changes.

This architecture will provide a strong foundation for building a reliable Construction ERP platform using modern development practices and AI-assisted development workflows.

End of Chapter 2: System Architecture Design

Chapter 3: Functional Requirements

3.1 Introduction

This chapter defines the functional requirements of the Construction Project Management & Accounting SaaS platform.

Functional requirements describe the features, behaviors, and operations that the system must provide to support construction business management activities.

The system will be developed as a modular platform where each module has specific responsibilities and business rules. These requirements will act as the foundation for database design, API development, UI implementation, and AI-assisted development.

The system should provide a complete workflow for managing construction projects, parties, employees, financial activities, documents, assets, and reports.

3.2 Authentication and User Access Management

3.2.1 User Authentication

The system shall provide secure authentication functionality for all users.

Users shall be able to:

- Register or be created by authorized users
- Login securely
- Logout from the system
- Reset password
- Update profile information

The authentication system should support future integration with:

- Email authentication
 - Phone authentication
 - Social login (optional)
-

3.2.2 User Management

The system shall allow authorized users to manage system users.

User information may include:

- Name
- Email
- Phone number
- Profile image
- Role
- Status
- Assigned permissions

The system shall support user activation and deactivation.

Inactive users shall not be able to access the system.

3.3 Role and Permission Management

3.3.1 Role Management

The system shall support role-based access control.

Default roles may include:

- Company Owner
- Administrator
- Accountant
- Project Manager
- Site Supervisor
- HR Manager

Users may have different permissions based on their assigned roles.

3.3.2 Permission Management

The system shall control access to modules and actions.

Permission levels may include:

- View
- Create
- Update
- Delete
- Approve
- Export

Example:

An Accountant may access:

- Finance
- Expense
- Payment
- Reports

A Site Supervisor may access:

- Attendance

- Site Update
 - Photo Upload
-

3.4 Company and Tenant Management

3.4.1 Company Profile

The system shall maintain company-level information.

Company information may include:

- Company name
 - Logo
 - Address
 - Contact information
 - Currency
 - Tax information
 - Business settings
-

3.4.2 Tenant Data Isolation

In SaaS mode, each company shall operate as an independent tenant.

The system must ensure:

- Tenant data separation
- Independent configuration
- Independent users
- Independent transactions

A user from one tenant must not be able to access another tenant's data.

3.5 Party Management Module

3.5.1 Party Concept

The system shall maintain a unified party management system.

A party may represent:

- Client
- Owner
- Supplier
- Subcontractor
- Consultant
- Other business relationships

A single party may have multiple roles.

3.5.2 Party Information

The system shall store party details including:

- Name
 - Company name
 - Contact person
 - Phone number
 - Email
 - Address
 - Party type
 - Notes
-

3.5.3 Party Transaction History

The system shall maintain complete financial history for each party.

The system shall display:

- Total transactions
- Received payments
- Paid amounts
- Outstanding balance
- Transaction history

3.6 Project Management Module

3.6.1 Project Creation

The system shall allow users to create multiple construction projects.

Project information may include:

- Project name
 - Project code
 - Client/Party
 - Location
 - Start date
 - Expected completion date
 - Project status
 - Description
-

3.6.2 Project Contract Management

The system shall support optional contract management.

Contract types may include:

- Lump Sum Contract
- Square Feet Contract
- Running Bill Contract
- Custom Contract

Contract information may include:

- Contract amount
- Contract date
- Payment terms
- Agreement documents

Contract information may be added during project creation or updated later.

3.6.3 Project Status Management

The system shall support project lifecycle tracking.

Possible statuses:

- Planning
 - Active
 - On Hold
 - Completed
 - Cancelled
-

3.7 Employee Management Module

3.7.1 Employee Profile Management

The system shall allow companies to maintain employee information.

Employee information may include:

- Name
- Phone
- Address
- Joining date
- Designation
- Skill category
- Salary type

Salary types:

- Daily Wage
 - Monthly Salary
 - Contract Based
-

3.7.2 Employee Project Assignment

The system shall allow employees to be assigned to specific projects.

The system shall track:

- Employee
 - Project
 - Assignment period
 - Role in project
-

3.7.3 Attendance Management

The system shall support employee attendance tracking.

Attendance information may include:

- Employee
 - Project
 - Date
 - Working status
 - Working hours
 - Overtime
-

3.7.4 Wage and Payroll Calculation

The system shall support both daily and monthly payroll calculation.

Daily wage calculation shall consider:

- Working days
- Daily rate
- Overtime
- Adjustment

Monthly salary calculation shall consider:

- Monthly salary
 - Attendance
 - Deduction
 - Advance payment
-

3.8 Subcontractor Management

3.8.1 Subcontractor Work Management

The system shall allow users to manage subcontractor activities.

Information may include:

- Subcontractor party
 - Assigned project
 - Work description
 - Contract amount
 - Work progress
-

3.8.2 Subcontractor Payment Management

The system shall track:

- Advance payment
 - Progress payment
 - Final payment
 - Outstanding balance
-

3.9 Finance Management Module

3.9.1 Income Management

The system shall record all project-related income.

Income sources may include:

- Client payment
 - Extra work payment
 - Other income
-

3.9.2 Expense Management

The system shall allow users to record project expenses.

Expense categories may include:

- Employee salary
 - Material expense
 - Transport
 - Equipment rental
 - Machinery cost
 - Utility expense
 - Other expenses
-

3.9.3 Transaction Management

All financial activities shall be stored as transactions.

Each transaction shall maintain:

- Transaction type
 - Amount
 - Date
 - Related project
 - Related party
 - Description
-

3.10 Machinery Management Module

The system shall support management of company-owned machinery.

Features include:

- Machinery registration
- Purchase information
- Purchase cost
- Current status
- Maintenance history
- Repair cost

- Fuel cost
 - Project usage history
-

3.11 Equipment Rental Management

The system shall support tracking rented equipment.

Information may include:

- Equipment name
 - Rental provider
 - Rental period
 - Rental rate
 - Start date
 - Return date
 - Payment history
 - Outstanding payment
-

3.12 Document Management

The system shall provide project-based document storage.

Supported documents may include:

- Contract agreements
- Site deal documents
- Design files
- Drawings
- Approval documents
- Site photos
- Progress photos

Documents will be optional and attached based on project requirements.

3.13 Notification Management

The system shall provide notification functionality based on business events.

Notifications may include:

- Payment due reminder
- Salary due reminder
- Project deadline reminder
- Equipment return reminder
- Important updates

The system should support future integration with:

- Email notification
 - SMS notification
 - Push notification
-

3.14 Reporting System

The system shall provide different types of reports.

Project Reports

- Project cost report
- Project income report
- Project expense report
- Project profitability report

Employee Reports

- Attendance report
- Wage report
- Salary payment report

Finance Reports

- Income report
- Expense report

- Cash flow report
- Party ledger report

Reports should support filtering by:

- Date range
 - Project
 - Party
 - Category
-

3.15 Functional Requirement Summary

The system shall provide a complete construction management solution covering project management, financial management, workforce management, document management, and reporting.

All modules shall be designed with future scalability and integration capability so that additional construction-related features can be added without major architectural changes.

End of Chapter 3: Functional Requirements

Chapter 4: Module Design Specification

4.1 Introduction

This chapter defines the detailed module design specification for the Construction Project Management & Accounting SaaS platform.

The system will follow a modular architecture where each module represents a specific business capability. Each module will have its own responsibility, business rules, data management, and workflow.

The purpose of this modular approach is to keep the system maintainable, scalable, and easy to extend. Future modules can be added without affecting the existing system functionality.

Each module will communicate with other modules through clearly defined interfaces and business events.

4.2 Core System Modules

Core modules provide common functionality required by all business modules.

The core modules include:

- Authentication Module
 - User Management Module
 - Role and Permission Module
 - Tenant Management Module
 - Company Settings Module
 - Notification Module
 - Audit Log Module
-

4.3 Authentication Module

4.3.1 Module Overview

The Authentication Module is responsible for managing user identity verification and secure access to the system.

This module will provide secure login functionality and manage user authentication throughout the application.

4.3.2 Main Responsibilities

The module shall handle:

- User login
 - User logout
 - Password management
 - Session management
 - Authentication security
 - Access token management
-

4.3.3 Future Extension

The module should support future authentication methods:

- Email verification
 - Phone verification
 - Two-factor authentication
 - Social login integration
-

4.4 Tenant Management Module

4.4.1 Module Overview

Tenant Management Module will manage company-level separation in the SaaS environment.

Each construction company using the system will be considered as a separate tenant.

4.4.2 Main Responsibilities

The module shall manage:

- Tenant creation
 - Company information
 - Tenant configuration
 - Tenant status
 - Data isolation
-

4.4.3 Tenant Data Isolation

The system must ensure that each tenant can only access its own:

- Users
- Projects
- Parties
- Employees
- Transactions

- Documents
 - Reports
-

4.5 User Management Module

4.5.1 Module Overview

User Management Module will manage all users associated with a company.

4.5.2 Main Responsibilities

The module shall support:

- User creation
 - User profile management
 - User activation/deactivation
 - User assignment
 - User information update
-

4.5.3 User Information

User profile may contain:

- Name
 - Email
 - Phone
 - Profile image
 - Role
 - Status
-

4.6 Role and Permission Module

4.6.1 Module Overview

The Role and Permission Module will provide access control throughout the application.

The purpose of this module is to ensure that users can only access features according to their responsibilities.

4.6.2 Permission Structure

The system shall support module-level and action-level permissions.

Permission actions may include:

- View
 - Create
 - Update
 - Delete
 - Approve
 - Export
-

4.6.3 Default Roles

The system may provide default roles:

Company Owner

Full access to company operations.

Administrator

Manage users, settings, and system configuration.

Accountant

Manage financial activities and reports.

Project Manager

Manage assigned projects and project operations.

Site Supervisor

Manage attendance, site updates, and photos.

4.7 Party Management Module

4.7.1 Module Overview

Party Management Module will maintain all external business relationships in a unified structure.

Instead of maintaining separate entities for client, owner, supplier, and subcontractor, the system will use a common Party entity.

4.7.2 Supported Party Types

A party may have one or multiple roles:

- Client
 - Owner
 - Supplier
 - Subcontractor
 - Consultant
 - Other
-

4.7.3 Main Responsibilities

The module shall manage:

- Party profile
 - Contact information
 - Party role
 - Transaction history
 - Outstanding balance
 - Related projects
-

4.8 Project Management Module

4.8.1 Module Overview

Project Management Module is one of the core business modules responsible for managing construction projects.

A company can create and manage multiple projects through this module.

4.8.2 Project Responsibilities

The module shall manage:

- Project creation
 - Project information
 - Client relationship
 - Project status
 - Timeline
 - Project documents
 - Project activities
-

4.8.3 Project Lifecycle

A project may follow the following lifecycle:

- Planning
 - Active
 - On Hold
 - Completed
 - Cancelled
-

4.9 Contract Management Module

4.9.1 Module Overview

Contract Management Module will manage project contract information.

Contract information will be optional and can be added during project creation or later.

4.9.2 Supported Contract Types

The system shall support:

- Lump Sum Contract
 - Square Feet Contract
 - Running Bill Contract
 - Custom Contract
-

4.9.3 Contract Information

The module shall maintain:

- Contract amount
 - Contract date
 - Payment terms
 - Agreement documents
 - Additional notes
-

4.10 Employee Management Module

4.10.1 Module Overview

Employee Management Module will manage company workforce information.

The module will support different types of construction workers and employees.

4.10.2 Employee Types

Supported employee payment models:

- Daily Wage Employee
 - Monthly Salary Employee
 - Contract Based Employee
-

4.10.3 Main Responsibilities

The module shall manage:

- Employee profile
 - Employee assignment
 - Attendance
 - Wage calculation
 - Salary history
-

4.11 Payroll Management Module

4.11.1 Module Overview

Payroll Module will handle employee payment calculation and payment records.

4.11.2 Payroll Features

The module shall support:

- Daily wage calculation
 - Monthly salary calculation
 - Overtime calculation
 - Advance payment
 - Deduction
 - Partial payment
 - Payment history
-

4.12 Subcontractor Management Module

4.12.1 Module Overview

Subcontractor Management Module will manage outsourced construction work and related payments.

4.12.2 Main Responsibilities

The module shall manage:

- Subcontractor assignment
 - Work details
 - Contract value
 - Progress tracking
 - Payment records
 - Outstanding balance
-

4.13 Finance Management Module

4.13.1 Module Overview

Finance Management Module will manage all financial activities of the construction business.

This module will act as the financial backbone of the system.

4.13.2 Main Responsibilities

The module shall manage:

- Income
- Expenses
- Payments
- Transactions
- Ledger
- Financial reports

4.13.3 Transaction Principle

All financial activities should be recorded as transactions.

Examples:

- Client payment received
 - Employee salary paid
 - Subcontractor payment made
 - Equipment rental paid
-

4.14 Machinery Management Module

4.14.1 Module Overview

Machinery Management Module will manage company-owned construction machinery and assets.

4.14.2 Main Responsibilities

The module shall manage:

- Machinery information
 - Purchase details
 - Asset value
 - Maintenance history
 - Repair cost
 - Fuel cost
 - Project usage history
-

4.15 Equipment Rental Management Module

4.15.1 Module Overview

This module will manage rented construction equipment.

4.15.2 Main Responsibilities

The module shall track:

- Equipment information
 - Rental provider
 - Rental period
 - Rental cost
 - Payment history
 - Return status
-

4.16 Document Management Module

4.16.1 Module Overview

Document Management Module will provide project-based file storage.

4.16.2 Supported Documents

The module shall support:

- Contract documents
 - Agreements
 - Drawings
 - Designs
 - Approval documents
 - Site photos
 - Progress photos
-

4.17 Notification Module

4.17.1 Module Overview

Notification Module will provide automated reminders and alerts based on business events.

4.17.2 Notification Types

The module shall support:

- Payment reminders
 - Salary reminders
 - Project deadline alerts
 - Equipment return reminders
 - System alerts
-

4.18 Reporting Module

4.18.1 Module Overview

Reporting Module will provide business insights through different reports.

4.18.2 Report Types

The module shall provide:

- Project reports
 - Financial reports
 - Employee reports
 - Party ledger reports
 - Expense reports
 - Profit and loss reports
-

4.19 Module Integration Principle

All modules should communicate through defined interfaces and business events.

Modules should avoid direct dependency on each other.

Example:

Employee Module should not directly update Finance Module data.

Instead:

Employee Payment Created Event

?

Finance Transaction Created

?

Report Updated

This approach will maintain system flexibility and scalability.

4.20 Module Design Summary

The Construction Project Management & Accounting SaaS platform will be developed using a modular architecture where each module has a clear responsibility.

This design will ensure:

- Easier maintenance
- Faster development
- Better AI-assisted coding
- Future module expansion
- Reduced system complexity

The module structure will provide a strong foundation for developing a complete Construction ERP platform.

End of Chapter 4: Module Design Specification

Chapter 5: Database Design

5.1 Introduction

The database design of Construction Project Management & Accounting SaaS is created to support scalability, data consistency, security, and future expansion.

The database structure will follow a modular approach where each business module will have clearly defined entities and relationships.

The primary objectives of the database design are:

- Maintain data integrity
- Support multi-tenant architecture
- Avoid data duplication
- Provide scalability for future growth
- Support complex construction business workflows
- Maintain clear relationships between business entities

The database will be designed based on relational database principles with proper normalization where required.

5.2 Database Design Principles

The following principles will be followed during database design.

5.2.1 Multi-Tenant Data Isolation

Since the system will support SaaS deployment, tenant-level data isolation is mandatory.

Most business tables will contain a tenant identifier to ensure that each company can access only its own data.

Example:

A construction company can access:

- Its own projects
- Its own employees
- Its own parties
- Its own transactions

A company cannot access another company's information.

5.2.2 Single Source of Truth

The database should maintain a single source of truth for important business information.

Example:

Party information will be stored once and reused across:

- Projects
- Finance
- Supplier management
- Subcontractor management

This approach will prevent duplicate records and maintain consistency.

5.2.3 Modular Data Structure

Each module will have its own database entities.

Example:

Project Module:

- Projects
- Contracts
- Project Documents

Employee Module:

- Employees
- Attendance
- Payroll

Finance Module:

- Transactions
 - Payments
 - Expenses
-

5.3 Core Database Entities

The core entities will support common system functionality.

Core entities include:

- Tenants
 - Users
 - Roles
 - Permissions
 - User Roles
 - Settings
 - Notifications
 - Audit Logs
-

5.4 Tenant Entity

Purpose

The Tenant entity represents each construction company using the SaaS platform.

Main Information

The entity will store:

- Company name
- Logo
- Address
- Contact information
- Configuration settings
- Status

Relationship:

One Tenant can have:

- Multiple Users
- Multiple Projects
- Multiple Parties

- Multiple Employees
 - Multiple Transactions
-

5.5 User Entity

Purpose

The User entity stores system users who can access the application.

Main Information

The entity will contain:

- Name
- Email
- Phone
- Password information
- Profile information
- Status
- Tenantreference

Relationship:

One Tenant can have multiple Users.

One User can have one or multiple Roles.

5.6 Role and Permission Entities

Purpose

These entities manage access control.

Role Entity

Stores:

- Role name
- Description
- Tenant reference

Examples:

- Owner
- Admin
- Accountant
- Project Manager
- Site Supervisor

Permission Entity

Stores:

- Module name
- Action name
- Permission description

Examples:

- Project Create
- Project Update
- Payment Approve
- Report Export

User Role Relationship

A user can have multiple roles depending on business requirements.

5.7 Party Management Entities

Party Entity

Purpose

Party entity represents all external business relationships.

A party can be:

- Client
- Owner
- Supplier
- Subcontractor
- Consultant

Main Fields

- TenantID
 - Name
 - Company Name
 - Contact Person
 - Phone
 - Email
 - Address
 - Notes
-

Party Type Entity

Stores different party roles.

Examples:

- Client
- Supplier
- Subcontractor

A single party may have multiple types.

Example:

A company can be both:

- Client
 - Supplier
-

5.8 Project Management Entities

Project Entity

Purpose

Stores construction project information.

Main Fields

- Tenant ID
- Project Name
- Project Code
- Party ID
- Location
- Start Date
- Expected End Date
- Status
- Description

Relationship:

One Project belongs to one Client/Party.

One Project can have:

- Multiple Contracts
- Multiple Employees
- Multiple Expenses
- Multiple Transactions
- Multiple Documents

Project Status Entity

Stores project lifecycle status.

Examples:

- Planning

- Active
 - On Hold
 - Completed
 - Cancelled
-

5.9 Contract Entities

Contract Entity

Purpose

Stores project agreement and contract information.

Main Fields

- Project ID
- Contract Type
- Contract Amount
- Contract Date
- Payment Terms
- Notes

Supported contract types:

- Lump Sum
 - Square Feet
 - Running Bill
 - Custom
-

5.10 Employee Management Entities

Employee Entity

Purpose

Stores employee information.

Main Fields

- Tenant ID
- Name
- Phone
- Address
- Designation
- Skill Type
- Salary Type

Salary types:

- Daily Wage
- Monthly Salary
- Contract Based

Employee Project Assignment Entity

Purpose:

Stores employee assignment history.

Information:

- Employee ID
- Project ID
- Assignment Date
- End Date

5.11 Attendance Entities

Attendance Entity

Purpose:

Stores employee attendance information.

Fields:

- Employee ID
 - Project ID
 - Date
 - Attendance Status
 - Working Hours
 - Overtime
-

5.12 Payroll Entities

Payroll Entity

Purpose:

Stores employee payment calculation.

Fields:

- Employee ID
 - Project ID
 - Salary Period
 - Total Amount
 - Paid Amount
 - Due Amount
 - Status
-

5.13 Subcontractor Entities

Subcontract Entity

Purpose:

Stores subcontractor work agreements.

Fields:

- Party ID
 - Project ID
 - Work Description
 - Contract Amount
 - Start Date
 - End Date
 - Status
-

5.14 Finance Entities

Transaction Entity

Purpose

Transaction entity will be the core financial entity of the system.

All financial activities will be stored as transactions.

Transaction types:

- Income
- Expense
- Payment
- Adjustment

Fields:

- Tenant ID
 - Project ID
 - Party ID
 - Transaction Type
 - Amount
 - Date
 - Description
-

Expense Entity

Purpose:

Stores project expenses.

Expense information:

- Project ID
 - Category
 - Amount
 - Date
 - Description
 - Attachment
-

5.15 Machinery Entities

Machinery Entity

Purpose:

Stores company-owned construction assets.

Fields:

- Name
 - Category
 - Purchase Date
 - Purchase Price
 - Current Status
 - Notes
-

Machinery Usage Entity

Purpose:

Tracks machinery usage by project.

Fields:

- Machinery ID
 - Project ID
 - Usage Date
 - Duration
 - Cost
-

5.16 Equipment Rental Entities

Equipment Rental Entity

Purpose:

Stores rented equipment information.

Fields:

- Project ID
 - Equipment Name
 - Rental Provider
 - Rental Rate
 - Rental Period
 - Payment Status
-

5.17 Document Management Entities

Document Entity

Purpose:

Stores project-related documents.

Document types:

- Contract
- Agreement
- Drawing
- Design

- Site Photo
- Other

Fields:

- Project ID
 - File Name
 - File Path
 - Document Type
 - Uploaded By
 - Upload Date
-

5.18 Notification Entities

Notification Entity

Purpose:

Stores system notifications.

Fields:

- User ID
 - Title
 - Message
 - Type
 - Status
 - Created Date
-

5.19 Audit Log Entity

Purpose

Stores important user activities for security and accountability.

Fields:

- User ID

- Action
 - Module
 - Record ID
 - Old Value
 - New Value
 - Timestamp
-

5.20 Database Relationship Overview

Major relationships:

Tenant

-> Users

-> Roles

-> Projects

-> Parties

-> Employees

-> Transactions

-> Documents

-> Reports

Project is the central business entity connecting:

- Client
 - Contract
 - Employee
 - Expense
 - Subcontractor
 - Machinery
 - Documents
 - Financial Transactions
-

5.21 Future Database Expansion

The database structure should support future modules without major changes.

Future entities may include:

- Materials
 - Inventory
 - Suppliers
 - Purchase Orders
 - BOQ Items
 - Tasks
 - Quality Checks
 - Safety Records
 - Client Portal Data
-

5.22 Database Design Summary

The database design of Construction Project Management & Accounting SaaS will provide a strong foundation for building a scalable construction ERP platform.

The design will ensure:

- Secure tenant isolation
- Clean entity relationships
- Accurate financial tracking
- Easy module expansion
- AI-friendly development structure

The database will support both current MVP requirements and future business growth.

End of Chapter 5: Database Design

Chapter 6: Business Rules & Workflow

6.1 Introduction

This chapter defines the core business rules and operational workflows of the Construction Project Management & Accounting SaaS platform.

Business rules describe how the system should process construction-related activities, maintain data consistency, and handle relationships between different modules.

The purpose of defining business rules is to ensure that the application behavior remains consistent across all modules and that AI-assisted development can follow a clear understanding of the business logic.

The system will follow an event-driven approach where important business actions will trigger related updates across connected modules.

6.2 General Business Rules

6.2.1 Data Ownership Rule

All business data must belong to a specific tenant.

Every business entity must maintain tenant ownership.

Examples:

- Projects belong to a company
- Employees belong to a company
- Transactions belong to a company
- Documents belong to a company

A user must only access data belonging to their assigned tenant.

6.2.2 Project-Centric Business Model

The project will be the central business entity of the system.

Most construction activities will be connected with a project.

A project may have:

- Client information
- Contract information

- Employees
- Subcontractors
- Expenses
- Income
- Documents
- Machinery usage

All project-related financial activities should be traceable through the project.

6.2.3 Transaction-Based Financial System

All financial activities must be recorded as transactions.

The system should avoid directly modifying financial balances.

Instead:

Transaction Created

?

Account Balance Updated

?

Reports Updated

This approach will maintain financial accuracy and auditability.

6.3 Project Management Business Rules

6.3.1 Project Creation Rule

A user with proper permission can create a project.

Minimum required information:

- Project Name
- Project Client/Party
- Project Status

Contract information is optional during project creation.

Users can add or update contract information later.

6.3.2 Project Contract Rule

A project may have different contract types.

Supported contract models:

Lump Sum Contract

The project has a fixed total contract amount.

Example:

Total project value = \$500,000

Square Feet Contract

The project value is calculated based on area measurement.

Example:

10,000 sq ft × rate per sq ft

Running Bill Contract

Payment is based on project progress or completed work.

Custom Contract

Allows businesses to define their own contract structure.

6.3.3 Project Status Workflow

A project should follow a controlled lifecycle.

Default workflow:

Planning

?

Active

?

Completed

Additional statuses:

- On Hold
- Cancelled

A completed project should not allow modification of important financial records without proper permission.

6.4 Client/Party Business Rules

6.4.1 Unified Party Management Rule

The system will use a single Party entity for all external relationships.

A party can have multiple roles.

Example:

A company can be:

- Client
- Supplier
- Subcontractor

The system should not create duplicate records for the same party.

6.4.2 Party Ledger Rule

Every financial transaction related to a party must update the party ledger.

Examples:

Client Payment Received:

?

Client Ledger Credit Updated

Subcontractor Payment Made:

?

6.5 Employee Management Business Rules

6.5.1 Employee Assignment Rule

An employee can work on multiple projects.

The system must maintain assignment history.

Example:

Employee A

Project X

January - March

Employee A

Project Y

April - June

6.5.2 Wage Calculation Rule

The system shall support multiple employee payment methods.

Supported methods:

- Daily Wage
 - Monthly Salary
 - Contract Based
-

6.5.3 Daily Wage Calculation Workflow

Daily wage calculation process:

Employee Attendance Recorded

?

Working Days Calculated

?

Daily Rate Applied

?

Overtime Added (If Applicable)

?

Adjustment Applied

?

Final Payment Generated

6.5.4 Monthly Salary Calculation Workflow

Monthly salary calculation process:

Salary Period Selected

?

Attendance Reviewed

?

Leave/Deduction Calculated

?

Advance Payment Adjusted

?

Final Salary Generated

6.6 Subcontractor Business Rules

6.6.1 Subcontract Assignment Rule

A subcontractor must be assigned to a specific project.

Each subcontract record must maintain:

- Subcontractor
 - Project
 - Work Description
 - Contract Amount
-

6.6.2 Subcontract Payment Rule

Subcontract payments must be tracked separately.

Payment types:

- Advance Payment
- Progress Payment
- Final Payment

Each payment must update:

- Project expense
 - Subcontractor ledger
 - Payment history
-

6.7 Finance Business Rules

6.7.1 Income Rule

All incoming money related to construction activities must be recorded as income transactions.

Examples:

- Client payment
- Additional work payment
- Other project income

Income transaction must be connected with:

- Project
- Party
- Date

- Amount
-

6.7.2 Expense Rule

All project-related costs must be recorded as expenses.

Expense categories may include:

- Employee payment
 - Subcontractor payment
 - Equipment rental
 - Machinery cost
 - Transportation
 - Other expenses
-

6.7.3 Project Profit Calculation Rule

Project profitability should be calculated using:

Project Profit = Total Income - Total Expense

Where:

Total Expense includes:

- Employee cost
 - Subcontractor cost
 - Equipment cost
 - Other expenses
-

6.8 Machinery and Equipment Business Rules

6.8.1 Owned Machinery Rule

Company-owned machinery should be treated as an asset.

The system should maintain:

- Purchase information

- Usage history
 - Maintenance cost
 - Repair history
-

6.8.2 Equipment Rental Rule

Rental equipment should not be treated as company assets.

The system should maintain:

- Rental provider
 - Rental period
 - Rental cost
 - Payment status
-

6.9 Document Management Rules

6.9.1 Document Attachment Rule

Documents should be linked with appropriate business entities.

Examples:

Contract Document

?

Project

Employee Document

?

Employee

Payment Receipt

?

Transaction

6.9.2 Document Access Rule

Document visibility should follow user permission rules.

Only authorized users should access sensitive documents.

6.10 Notification Business Rules

6.10.1 Event-Based Notification Rule

Notifications should be generated based on business events.

Examples:

Payment Due Date Approaching

?

Payment Reminder Notification

Salary Date Reached

?

Salary Reminder Notification

Project Deadline Nearing

?

Project Alert Notification

6.11 Audit and History Rules

6.11.1 Important Action Tracking

The system should record important user activities.

Tracked activities:

- Create
- Update

- Delete
 - Approve
 - Payment changes
 - Settings changes
-

6.11.2 Financial Record Protection

Financial transactions should not be permanently deleted after confirmation.

Instead:

- Void
- Cancel
- Adjustment

methods should be used.

This will maintain financial history.

6.12 Event-Driven Workflow Examples

Client Payment Workflow

Client Payment Created

?

Income Transaction Created

?

Party Ledger Updated

?

Project Income Updated

?

Project Profit Report Updated

Employee Payment Workflow

Employee Salary Generated

?

Payment Transaction Created

?

Employee Payment History Updated

?

Project Cost Updated

?

Financial Report Updated

Equipment Rental Workflow

Equipment Rental Added

?

Rental Expense Created

?

Project Expense Updated

?

Payment Due Notification Generated

6.13 Business Rule Summary

The system will follow a structured business rule framework where all construction activities are connected through projects, parties, and financial transactions.

The use of event-driven workflows will ensure:

- Accurate financial tracking
- Better data consistency

- Easy reporting
- Reduced module dependency
- Future scalability

These business rules will act as the foundation for application development, database implementation, and AI-assisted coding.

End of Chapter 6: Business Rules & Workflow

Chapter 7: Security & Permission Design

7.1 Introduction

Security is a critical part of Construction Project Management & Accounting SaaS because the system will handle sensitive business information including financial transactions, employee records, project details, contracts, documents, and company data.

The security design will ensure that business data remains protected, users can access only authorized information, and all important activities are properly tracked.

The system will follow a layered security approach including:

- Authentication security
- Authorization control
- Tenant data isolation
- Data protection
- Audit tracking
- Secure file management
- Financial data protection

The security architecture will support both SaaS deployment and self-hosted deployment models.

7.2 Security Design Principles

7.2.1 Principle of Least Privilege

Users should only receive the minimum permissions required to perform their responsibilities.

Example:

An accountant should not automatically have permission to modify system settings.

A site supervisor should not access complete financial reports unless permission is granted.

7.2.2 Role-Based Access Control (RBAC)

The system will use Role-Based Access Control to manage user permissions.

Access will be controlled based on:

- User role
 - Module permission
 - Action permission
 - Tenant access
-

7.2.3 Data Isolation

Since the platform supports SaaS architecture, tenant data isolation is mandatory.

The system must ensure:

- One company cannot view another company's data
 - Queries are always filtered by tenant context
 - Files are isolated between tenants
 - Reports only contain tenant-specific information
-

7.3 Authentication Security

7.3.1 User Authentication

The system shall provide secure authentication mechanisms.

Authentication responsibilities include:

- User identity verification
- Secure password handling
- Session management
- Access token management

7.3.2 Password Security

The system shall follow secure password practices.

Requirements:

- Passwords must never be stored as plain text
 - Passwords must be securely hashed
 - Minimum password rules should be configurable
 - Password reset process must be secure
-

7.3.3 Session Management

The system shall securely manage user sessions.

Security measures:

- Token expiration
 - Session invalidation after logout
 - Multiple device session control (optional)
 - Unauthorized session detection
-

7.4 Authorization Design

7.4.1 Permission Structure

The system will support fine-grained permission management.

Permission structure:

Module

?

Feature

?

Action

Example:

Project Module

- View Project
- Create Project
- Update Project
- Delete Project

Finance Module

- View Transaction
 - Create Payment
 - Approve Payment
 - Export Report
-

7.4.2 Default Roles

The system may provide default roles.

Company Owner

Permissions:

- Full company access
- Financial overview
- User management
- Settings management

Administrator

Permissions:

- User management
- Permission management
- System configuration

Accountant

Permissions:

- Income management
- Expense management
- Payment records
- Financial reports

Project Manager

Permissions:

- Project management
- Employee assignment
- Project documents
- Project reports

Site Supervisor

Permissions:

- Attendance update
 - Site photo upload
 - Site progress update
-

7.5 Tenant Security

7.5.1 Tenant Context Validation

Every request must validate tenant identity before processing.

The system must verify:

- User belongs to tenant
 - Requested data belongs to same tenant
 - User permission exists
-

7.5.2 Tenant Data Protection

The system should prevent:

- Cross-tenant data access
 - Unauthorized data modification
 - Data leakage through APIs
 - File access outside tenant scope
-

7.6 API Security Design

7.6.1 API Authentication

All protected APIs must require authentication.

The system should validate:

- Access token
 - User identity
 - Tenant identity
 - Permission
-

7.6.2 API Authorization

Each API request must verify whether the user has permission for the requested operation.

Example:

Creating a project:

Request

?

Authentication Check

?

Tenant Check

?

Permission Check

?

Business Validation

?

Process Request

7.6.3 Input Validation

All user inputs must be validated before processing.

Validation should prevent:

- Invalid data
 - SQL injection
 - Malicious input
 - Incorrect financial values
-

7.7 Financial Data Security

7.7.1 Transaction Protection

Financial records require special protection.

The system should:

- Track transaction history
 - Prevent unauthorized deletion
 - Maintain audit records
 - Support transaction cancellation instead of removal
-

7.7.2 Approval Workflow

Sensitive financial operations may require approval.

Examples:

- Large payments
- Expense approval
- Refund approval

Approval flow:

Created

?

Pending Approval

?

Approved / Rejected

7.8 Document Security

7.8.1 File Access Control

Project documents may contain sensitive information.

The system shall control access based on:

- Tenant
 - Project access
 - User permission
-

7.8.2 Secure File Storage

Uploaded files should be stored securely.

Security requirements:

- Unique file identification
 - Access-controlled download
 - File type validation
 - File size limitation
-

7.9 Audit Logging

7.9.1 Audit Purpose

Audit logging will provide transparency and accountability.

The system shall record important user activities.

7.9.2 Audited Activities

The system should track:

- User login
 - User logout
 - Project creation
 - Project update
 - Payment creation
 - Payment modification
 - Expense changes
 - Permission changes
 - Settings changes
-

7.9.3 Audit Information

Audit records should contain:

- User
 - Action
 - Module
 - Record ID
 - Previous value
 - New value
 - Timestamp
 - IP information (optional)
-

7.10 Data Protection

7.10.1 Data Encryption

Sensitive information should be protected using encryption where required.

Sensitive data may include:

- Password information
 - Financial information
 - Private documents
-

7.10.2 Backup Strategy

The system should support regular backup mechanisms.

Backup should include:

- Database backup
- Document backup
- Configuration backup

Backup strategy may differ between:

- SaaS deployment
 - Self-hosted deployment
-

7.11 Security Monitoring

The system should support security monitoring features.

Monitoring areas:

- Failed login attempts
 - Suspicious activities
 - Unauthorized access attempts
 - System errors
-

7.12 Notification Security

Notifications should not expose sensitive information unnecessarily.

Example:

Instead of:

"Payment of \$50,000 pending for ABC Project"

Use:

"Payment reminder available. Please check the system."

Detailed information should only be visible after proper authentication.

7.13 Security Rules for AI Development

Since the system will be developed using AI-assisted development tools, security rules must be clearly defined.

AI-generated code must follow:

- No hardcoded credentials
 - Secure environment variable usage
 - Proper authentication checks
 - Permission validation
 - Input sanitization
 - Secure database queries
 - Proper error handling
-

7.14 Security Design Summary

The security architecture of Construction Project Management & Accounting SaaS will ensure protection of business data, financial records, and user information.

The system will implement:

- Secure authentication
- Role-based permissions
- Tenant isolation
- API security
- Financial protection
- Audit tracking
- Secure document management

This security framework will provide a reliable foundation for building a scalable and trusted Construction ERP platform.

Chapter 8: Deployment Architecture

8.1 Introduction

The deployment architecture of Construction Project Management & Accounting SaaS is designed to support flexibility, scalability, reliability, and easy maintenance.

The system will support two major deployment models:

1. SaaS Cloud Deployment
2. Self-Hosted Deployment

The same application architecture should be capable of supporting both deployment models with minimal configuration changes.

The deployment strategy will focus on:

- Easy installation
 - Secure data management
 - Scalability
 - Cost optimization
 - High availability
 - Simple maintenance
-

8.2 Deployment Architecture Overview

The system will follow a modern web application deployment architecture consisting of multiple layers.

High-level architecture:

User

?

Web Browser / Mobile Browser

?

Frontend Application

?

Backend API Server

?

Database Server

?

File Storage

?

External Services

Each layer will have a specific responsibility to improve scalability and maintainability.

8.3 Application Components

The system will consist of the following major components:

8.3.1 Frontend Application

The frontend application will provide the user interface for all system operations.

Responsibilities:

- User interface rendering
- Dashboard display
- Form handling
- Data visualization
- Report viewing
- Responsive mobile experience

The frontend should be developed as a responsive web application so that users can access the system from:

- Desktop
- Tablet
- Mobile browser

The same responsive application can later be converted into a mobile application using technologies such as:

- Progressive Web App (PWA)

- WebView wrapper
 - Mobile application framework
-

8.3.2 Backend Application

The backend application will provide business logic and API services.

Responsibilities:

- Authentication
- Authorization
- Business rules
- Data processing
- Transaction handling
- Report generation
- File management

The backend should follow modular architecture where each business module has separate responsibilities.

8.3.3 Database Server

The database server will store all business and system information.

Data includes:

- Users
- Companies
- Projects
- Parties
- Employees
- Transactions
- Documents
- Reports

The database should support:

- Data integrity
- Transaction management
- Backup

- Scalability
-

8.3.4 File Storage

Construction projects require storing different types of files.

File storage will manage:

- Site photos
- Contract documents
- Drawings
- Designs
- Agreements
- Reports

The storage system should support:

- Secure upload
 - Access control
 - File organization
 - Backup
-

8.4 SaaS Deployment Architecture

8.4.1 Overview

In SaaS deployment, multiple construction companies will use the same application platform.

The system will operate as a multi-tenant application.

Each tenant will have:

- Separate company profile
 - Separate users
 - Separate projects
 - Separate financial records
 - Separate documents
-

8.4.2 SaaS Infrastructure Components

A typical SaaS deployment will include:

Frontend Server

?

Backend API Server

?

Database Server

?

File Storage Service

Additional services:

- Email service
- Notification service
- Monitoring service
- Backup service

8.5 Self-Hosted Deployment Architecture

8.5.1 Overview

The system should support deployment on a customer's own server or cloud infrastructure.

A self-hosted customer will have:

- Own domain
- Own application instance
- Own database
- Own file storage

8.5.2 Self-Hosted Benefits

Self-hosted deployment provides:

- Full data ownership
 - Custom environment
 - Independent operation
 - Internal security control
-

8.6 Environment Structure

The system should maintain separate environments for development and production.

Required environments:

Development Environment

Purpose:

- Feature development
- Testing
- AI-assisted coding
- Debugging

Testing Environment

Purpose:

- Quality assurance testing
- Integration testing
- User acceptance testing

Production Environment

Purpose:

- Live customer usage
 - Real business data
 - High availability
-

8.7 Deployment Workflow

The deployment process should follow a controlled workflow.

Development

?

Code Review

?

Automated Testing

?

Build Process

?

Deployment

?

Monitoring

The system should support automated deployment processes in the future.

8.8 Container-Based Deployment

The application should support containerized deployment.

Containerization benefits:

- Consistent environment
- Easy deployment
- Better resource management
- Simple scaling

The deployment may include containers for:

- Frontend
- Backend API
- Database
- Background workers

- File services
-

8.9 Cloud Deployment Strategy

The system can be deployed on different cloud providers.

Possible platforms:

- AWS
- Google Cloud
- Azure
- DigitalOcean
- Railway
- Render
- Other cloud providers

The architecture should avoid unnecessary dependency on a single provider.

8.10 Domain and Branding Support

For SaaS deployment:

The system may support:

- Main application domain
- Tenant-specific branding
- Company logo
- Company information

For self-hosted deployment:

The system should support:

- Custom domain
 - SSL certificate
 - Custom branding
-

8.11 Database Deployment Strategy

The database deployment strategy should support both SaaS and self-hosted models.

Possible approaches:

Shared Database Model

Multiple tenants use the same database with tenant isolation.

Advantages:

- Lower infrastructure cost
- Easier maintenance

Separate Database Model

Each tenant has an independent database.

Advantages:

- Better isolation
- Easier backup and migration

The final approach can be selected based on business scale and deployment requirements.

8.12 Backup and Recovery Strategy

The system should provide backup and recovery mechanisms.

Backup should include:

- Database backup
- Uploaded document backup
- Configuration backup

Recovery process should support:

- Data restoration
 - Disaster recovery
 - Backup verification
-

8.13 Monitoring and Logging

The deployment environment should support monitoring.

Monitoring areas:

- Application performance
- Server resources
- Database performance
- Error tracking
- Security events

Logs should include:

- Application logs
 - API logs
 - Error logs
 - Audit logs
-

8.14 Scaling Strategy

The architecture should support horizontal and vertical scaling.

Vertical Scaling

Increasing server resources:

- CPU
- RAM
- Storage

Horizontal Scaling

Adding more application instances.

Examples:

- Multiple backend servers
- Load balancing
- Distributed services

8.15 Mobile Application Strategy

The system will be developed as a responsive web application first.

The same application can support mobile users through:

- Responsive design
- PWA support
- WebView mobile application

A native mobile application can be introduced later if business requirements demand it.

8.16 Deployment Security Requirements

Deployment environment must ensure:

- HTTPS communication
 - Secure environment variables
 - Database access protection
 - Regular updates
 - Firewall configuration
 - Access monitoring
-

8.17 Deployment Architecture Summary

The deployment architecture of Construction Project Management & Accounting SaaS will provide a flexible foundation for both SaaS and self-hosted business models.

The architecture will support:

- Cloud deployment
- Private deployment
- Mobile access
- Future scaling
- Easy maintenance
- Secure operation

This deployment approach will allow the platform to grow from an MVP into a complete Construction ERP

solution.

End of Chapter 8: Deployment Architecture

Chapter 9: AI Development Guidelines

9.1 Introduction

Construction Project Management & Accounting SaaS will be developed using AI-assisted development tools to improve development speed, maintain code quality, and reduce repetitive implementation effort.

Since AI coding assistants work based on available context, clear documentation, structured requirements, and well-defined development rules are essential.

This chapter defines the guidelines for using AI tools during the development process.

The purpose of these guidelines is to ensure that AI-generated code remains:

- Consistent
 - Maintainable
 - Secure
 - Scalable
 - Aligned with project architecture
-

9.2 AI Development Principles

9.2.1 Documentation First Development

Before implementing any feature, the related requirement, business rule, database structure, and workflow should be clearly documented.

AI should generate code based on approved documentation rather than assumptions.

Development flow:

Requirement Definition

?

Business Rule Definition

?

Database Design

?

API Design

?

Implementation

?

Testing

9.2.2 Modular Development Approach

The project must be developed module by module.

Each module should have:

- Clear responsibility
- Independent business logic
- Defined database entities
- Defined API endpoints
- Defined permission rules

Example:

Project Module

- Project entity
 - Project service
 - Project API
 - Project validation
 - Project permission
-

9.3 AI Prompt Structure Guidelines

AI prompts should be structured clearly to get consistent output.

A good development prompt should include:

Context

Explain the project and current module.

Example:

"You are working on the Construction Project Management & Accounting SaaS platform."

Task

Clearly define what needs to be developed.

Example:

"Create the Project Management module with CRUD operations."

Technical Requirements

Mention:

- Technology stack
- Architecture pattern
- Database requirements
- Coding standards

Business Rules

Provide related business logic.

Example:

"A project must belong to a tenant and a client party."

Expected Output

Define what AI should generate.

Example:

"Generate database schema, API design, service layer, and validation rules."

9.4 AI Context Management Strategy

AI coding tools have limited context capacity.

To reduce unnecessary token usage, project information should be organized properly.

The AI should not receive the entire project documentation for every task.

Instead, provide only required context.

Example:

For Employee Module:

Required Context:

- Employee requirements
- Payroll rules
- Database structure
- Permission rules

Unnecessary Context:

- Machinery module
- Equipment rental module
- Notification details

9.5 Project Documentation Structure for AI

The project documentation should maintain a structured format.

Recommended structure:

```
/docs

01-project-overview.md

02-system-architecture.md

03-functional-requirements.md

04-module-design.md

05-database-design.md
```

06-business-rules.md

07-security-design.md

08-deployment.md

09-ai-guidelines.md

AI assistants should refer to these documents during development.

9.6 AI Development Workflow

The recommended AI-assisted development workflow:

Step 1: Requirement Analysis

Understand the feature requirement.

Example:

Employee wage calculation.

Step 2: Business Rule Definition

Define how the feature should behave.

Example:

Daily wage = Working days × Daily rate.

Step 3: Database Planning

Define required entities and relationships.

Step 4: API Design

Define:

- Endpoint
- Request structure
- Response structure
- Validation rules

Step 5: Implementation

Generate code according to approved design.

Step 6: Review and Testing

AI-generated code must be reviewed and tested before acceptance.

9.7 AI Coding Rules

AI-generated code must follow the following rules:

Clean Code

Code should be:

- Readable
 - Organized
 - Properly named
 - Easy to maintain
-

No Duplicate Logic

Common functionality should be reusable.

Example:

Payment calculation logic should not be duplicated in multiple modules.

Proper Error Handling

Every module should handle:

- Validation errors
 - Business rule errors
 - System errors
-

Secure Coding

AI must avoid:

- Hardcoded passwords
 - Hardcoded API keys
 - Unsafe queries
 - Missing permission checks
-

9.8 AI Database Development Rules

When generating database-related code, AI must consider:

- Proper relationships
- Foreign keys
- Data validation
- Tenant isolation
- Indexing requirements

Every business table should consider:

Common fields:

- id
- tenant_id
- created_at
- updated_at
- created_by

Where applicable.

9.9 AI API Development Rules

All APIs should follow consistent standards.

Each API should define:

- Authentication requirement
- Permission requirement
- Request validation

- Response format
- Error handling

Example:

Create Project API:

Authentication Check

?

Permission Check

?

Input Validation

?

Business Rule Validation

?

Create Project

?

Return Response

9.10 AI Frontend Development Rules

Frontend development should follow:

- Component-based architecture
- Reusable UI components
- Consistent design system
- Responsive layout

The interface should support:

- Desktop
- Tablet
- Mobile

Since the application will be used through mobile browsers, mobile-first responsive design is required.

9.11 AI Testing Guidelines

AI should assist in generating tests for important functionality.

Testing should include:

Unit Testing

For:

- Business logic
- Calculations
- Validation

Integration Testing

For:

- API workflow
- Database operations
- Module communication

End-to-End Testing

For:

- Complete user workflows

Example:

Create Project

?

Add Employee

?

Record Attendance

?

Generate Salary

?

Create Payment

?

View Report

9.12 AI Code Review Checklist

Before accepting AI-generated code, review:

Architecture

- Does it follow project architecture?
- Is module separation maintained?

Security

- Are permissions checked?
- Is data protected?

Database

- Are relationships correct?
- Is tenant isolation maintained?

Business Logic

- Does it follow documented rules?

Maintainability

- Is the code reusable?
 - Is unnecessary complexity avoided?
-

9.13 Avoiding AI Development Problems

Common AI development problems should be avoided.

Over Engineering

AI should not create unnecessary complexity.

The implementation should match current project requirements.

Unnecessary Dependencies

Avoid adding libraries without clear benefits.

Every dependency should have a purpose.

Inconsistent Code Style

AI-generated code must follow existing project conventions.

Ignoring Business Rules

AI should not implement features without understanding business workflows.

9.14 AI-Assisted Feature Development Example

Example: Employee Payment Feature

Required Context:

- Employee module
- Payroll rules
- Finance rules
- Database entities

Prompt:

"Implement employee payment workflow according to payroll and finance business rules. When payment is created, generate a finance transaction and update employee payment history."

Expected Result:

- Database changes

- Backend logic
 - API implementation
 - Frontend interface
 - Tests
-

9.15 Future AI Development Strategy

As the project grows, AI usage should evolve with proper documentation and reusable prompts.

Future improvements:

- Maintain module-specific AI prompts
 - Maintain coding standards document
 - Maintain reusable development templates
 - Automate testing generation
 - Automate documentation updates
-

9.16 AI Development Guidelines Summary

AI will be used as a development assistant, not as a replacement for architectural decisions.

The success of AI-assisted development depends on:

- Clear requirements
- Proper documentation
- Structured prompts
- Modular architecture
- Continuous review

Following these guidelines will allow faster development while maintaining the quality, security, and scalability of the Construction Project Management & Accounting SaaS platform.

End of Chapter 9: AI Development Guidelines

Chapter 10: Development Roadmap & Implementation Strategy

10.1 Introduction

This chapter defines the recommended development roadmap and implementation strategy for Construction Project Management & Accounting SaaS.

The purpose of this roadmap is to build the system in a structured way by prioritizing core business requirements first and gradually introducing advanced features.

Since the platform is designed as a scalable Construction ERP solution, development should follow an incremental approach instead of attempting to build every feature at the beginning.

The implementation strategy will focus on:

- Building a stable foundation
 - Delivering a usable MVP
 - Validating business requirements
 - Reducing development complexity
 - Supporting future expansion
-

10.2 Development Approach

The project will follow an iterative development approach.

Each development phase will include:

- Requirement analysis
- Database design
- Backend implementation
- Frontend implementation
- Testing
- Review
- Documentation update

The system should be developed module by module while maintaining consistency with the overall architecture.

10.3 Phase 1: Foundation Development

Objective

Create the core system foundation required for all future modules.

Modules Included

Authentication Module

Features:

- User login
- Logout
- Password management
- Session management

Tenant Management

Features:

- Company creation
- Company profile
- Tenant configuration
- Data isolation

User Management

Features:

- User creation
- User profile
- User status management

Role and Permission Management

Features:

- Role creation
- Permission assignment
- Access control

Basic Settings

Features:

- Company information
- Currency settings
- General configuration

Expected Outcome

A secure and scalable application foundation ready for business module development.

10.4 Phase 2: Core Construction Management MVP

Objective

Develop the first usable version focused on construction project operations.

Modules Included

Party Management

Features:

- Client management
- Owner management
- Supplier management
- Subcontractor management
- Party history

Project Management

Features:

- Create project
- Update project
- Project status tracking
- Project information management

Contract Management

Features:

- Lump sum contract
- Square feet contract
- Custom contract
- Contract documents

Document Management

Features:

- Upload documents
- Store project files
- Manage site photos

Expected Outcome

Users can create projects and manage basic construction information.

10.5 Phase 3: Workforce and Payment Management

Objective

Introduce employee and payment-related functionality.

Modules Included

Employee Management

Features:

- Employee profile
- Employee assignment
- Employee history

Attendance Management

Features:

- Daily attendance
- Project-based attendance
- Working hours tracking

Payroll Management

Features:

- Daily wage calculation
- Monthly salary calculation
- Advance payment
- Payment history

Subcontractor Management

Features:

- Subcontractor assignment
- Work tracking
- Payment management

Expected Outcome

Construction businesses can manage workforce operations and employee/subcontractor payments.

10.6 Phase 4: Financial Management

Objective

Develop complete project-based accounting functionality.

Modules Included

Income Management

Features:

- Client payment entry
- Payment history
- Outstanding tracking

Expense Management

Features:

- Project expenses
- Expense categories
- Expense history

Transaction Management

Features:

- Income transaction
- Expense transaction
- Payment transaction
- Ledger

Financial Reports

Features:

- Project income report
- Expense report
- Profit/loss report

Expected Outcome

Users can understand the financial condition of each project.

10.7 Phase 5: Asset and Equipment Management

Objective

Manage construction assets and equipment usage.

Modules Included

Machinery Management

Features:

- Machinery records
- Purchase information
- Maintenance history
- Usage tracking

Equipment Rental Management

Features:

- Rental equipment records
- Rental cost tracking
- Payment history
- Return tracking

Expected Outcome

Businesses can monitor equipment-related costs and assets.

10.8 Phase 6: Advanced Reporting and Analytics

Objective

Provide better business intelligence.

Features:

- Project profitability dashboard
- Expense analysis
- Payment analysis
- Employee cost analysis
- Business overview dashboard

Future analytics may include:

- Cost prediction

- Project performance comparison
 - Automated insights
-

10.9 Phase 7: Mobile Experience Enhancement

Objective

Improve mobile usability for field workers and management.

Initial approach:

- Responsive web application
- Mobile-first UI design

Future expansion:

- Progressive Web App
- Android application
- iOS application

Mobile features may include:

- Attendance update
 - Site photo upload
 - Project status update
 - Notification viewing
-

10.10 Phase 8: Advanced Construction ERP Modules

Future modules may include:

Inventory Management

Features:

- Material stock
- Stock movement

- Material usage tracking

Procurement Management

Features:

- Purchase requests
- Supplier management
- Purchase orders

BOQ Management

Features:

- Bill of Quantity
- Cost estimation
- Material planning

Client Portal

Features:

- Client project view
- Payment history
- Project updates

10.11 Implementation Priority

Development priority should follow business value.

Priority Order:

1. Authentication and Security
2. Tenant Management
3. Party Management
4. Project Management
5. Employee Management
6. Payroll Management
7. Finance Management

8. Reporting
9. Asset Management
10. Advanced ERP Modules

This sequence ensures that the system becomes usable early while maintaining a scalable foundation.

10.12 MVP Definition

The first MVP version should include:

Required Modules

- Authentication
- User Management
- Role Permission
- Company Management
- Party Management
- Project Management
- Employee Management
- Attendance
- Payroll
- Expense Management
- Income Management
- Basic Reports

Optional MVP Features

- Document Upload
- Site Photos
- Notifications

The MVP should focus on solving the most important problems of construction owners.

10.13 Development Quality Standards

Every development phase must maintain:

- Clean architecture
- Proper testing
- Security validation
- Documentation updates
- Code review

Before completing a module:

Required:

- Database review
- API review
- Permission review
- Business rule validation
- Testing

10.14 AI-Assisted Development Strategy

AI development should follow the documented roadmap.

For each module:

Step 1:

Provide module requirements.

Step 2:

Provide related business rules.

Step 3:

Generate database design.

Step 4:

Generate backend implementation.

Step 5:

Generate frontend implementation.

Step 6:

Generate tests.

Step 7:

Review and improve.

This approach will reduce AI errors and maintain consistent development quality.

10.15 Final Development Goal

The ultimate goal is to transform Construction Project Management & Accounting SaaS into a complete Construction ERP platform.

The system should provide construction businesses with a reliable solution for:

- Project management
- Financial control
- Workforce management
- Asset management
- Reporting
- Business decision-making

The development roadmap ensures that the platform can start as a practical MVP and gradually evolve into a powerful enterprise-level construction management system.

End of Chapter 10: Development Roadmap & Implementation Strategy

Chapter 11: Technical Architecture & Technology Stack

11.1 Introduction

This chapter defines the technical architecture and recommended technology stack for the Construction Project Management & Accounting SaaS platform.

The system architecture should support:

- SaaS deployment
- Self-hosted deployment
- Mobile-responsive usage
- Future module expansion

- AI-assisted development
- Long-term maintainability

The architecture should avoid unnecessary complexity while maintaining scalability.

The primary goal is to build a reliable MVP foundation that can gradually evolve into a complete Construction ERP platform.

11.2 Architecture Approach

The system will follow a modular layered architecture.

The architecture will separate:

- User Interface Layer
- Application Logic Layer
- Data Access Layer
- Infrastructure Layer

High-level architecture:

```
Client Applications
(Web Browser / Mobile Browser)
    ?
Frontend Application
    ?
Backend API Application
    ?
Business Modules
    ?
Database & Storage Layer
```

11.3 Frontend Architecture

11.3.1 Frontend Responsibilities

The frontend application will handle:

- User interface
- User interaction
- Form management
- Dashboard visualization
- Report presentation
- Responsive mobile experience

The frontend should be designed as a reusable component-based system.

11.3.2 Frontend Requirements

The frontend must support:

- Desktop browsers
- Tablet devices
- Mobile browsers

The design approach should follow:

- Mobile-first responsive design
 - Reusable components
 - Consistent UI patterns
 - Accessibility principles
-

11.3.3 Recommended Frontend Technology

Recommended stack:

- Next.js
- React
- TypeScript
- Tailwind CSS

- Component library

Reasons:

- Strong ecosystem
- AI-friendly development
- Good performance
- SEO support
- Easy deployment

11.4 Backend Architecture

11.4.1 Backend Responsibilities

The backend application will handle:

- Authentication
- Business rules
- Data processing
- API communication
- Permission validation
- Financial calculations
- Report generation

11.4.2 Backend Design Pattern

The backend should follow modular architecture.

Example:

Backend

??? Authentication Module

??? User Module

??? Party Module

??? Project Module

??? Employee Module

??? Finance Module

??? Report Module

??? Notification Module

Each module should contain:

- Controller/API layer
- Service layer
- Data access layer
- Validation rules
- Business logic

11.5 Recommended Backend Technology

Recommended options:

Option 1: NestJS + TypeScript

Advantages:

- Enterprise-level structure
- Modular architecture
- Strong TypeScript support
- Good scalability
- AI coding friendly

Option 2: Laravel

Advantages:

- Rapid development
- Strong ecosystem
- Easy CRUD development
- Good for business applications

The final selection should consider:

- Development speed
 - Long-term maintenance
 - AI tool support
 - Developer experience
-

11.6 Database Architecture

11.6.1 Database Requirements

The database must support:

- Complex relationships
 - Financial transactions
 - Reporting queries
 - Data integrity
 - Multi-tenant architecture
-

11.6.2 Recommended Database

Primary database:

- PostgreSQL

Reasons:

- Strong relational support
 - Excellent transaction handling
 - JSON support
 - Scalability
 - Open-source
-

11.7 ORM and Data Access Layer

The system should use an ORM to simplify database operations.

Possible options:

For TypeScript:

- Prisma
- TypeORM

For Laravel:

- Eloquent ORM

Benefits:

- Faster development
- Better maintainability
- Reduced SQL complexity
- Type safety

11.8 API Architecture

11.8.1 API Style

The system should use REST API architecture initially.

Reasons:

- Simple implementation
- Easy integration
- Mobile application support
- Third-party integration support

11.8.2 API Structure Example

```
/api/v1/projects  
  
/api/v1/employees  
  
/api/v1/payments  
  
/api/v1/reports
```

API versioning should be maintained from the beginning.

11.9 File Storage Architecture

Construction applications require storing many files.

Examples:

- Site photos
- Contracts
- Designs
- Documents

The system should separate file storage from the database.

Database stores:

- File information
- File location
- Access permission

Storage stores:

- Actual files

11.10 Recommended File Storage Options

Possible options:

- AWS S3
- Cloudflare R2
- DigitalOcean Spaces
- Self-hosted storage

The system should support changing storage providers without major code changes.

11.11 Background Processing Architecture

Some tasks should run asynchronously.

Examples:

- Report generation
- Notification sending
- File processing
- Backup tasks

The system may use:

- Queue system
- Background workers
- Scheduled jobs

11.12 Notification Architecture

Notification system should be designed as an independent module.

Future support:

- Email
- SMS
- Push notification
- WhatsApp integration

Notification flow:

Business Event

?

Notification Service

?

User Notification

11.13 Multi-Tenant Architecture Design

The system will support SaaS multi-tenancy.

Possible approaches:

Shared Database

All tenants use one database with tenant isolation.

Advantages:

- Lower cost
- Easier maintenance

Separate Database

Each tenant has a separate database.

Advantages:

- Strong isolation
- Easier customer migration

The architecture should allow both approaches if future business requirements change.

11.14 Self-Hosted Architecture Support

The system should be deployable independently.

Self-hosted deployment should support:

- Custom domain
- Custom database
- Custom storage
- Independent configuration

The application should avoid hard dependency on cloud providers.

11.15 Mobile Application Strategy

The first release will focus on responsive web application development.

The application should support:

- Mobile browser usage
- PWA installation

- WebView packaging

Future:

Native Android/iOS application can be developed using:

- React Native
 - Flutter
-

11.16 Development Environment

Recommended development setup:

Local Development

Includes:

- Source code repository
- Local database
- Environment configuration
- Testing tools

Version Control

The project should use:

- Git
 - Branch-based workflow
 - Code review process
-

11.17 Technology Selection Principles

Technology decisions should follow:

- Simplicity
- Maintainability
- Cost efficiency
- Developer productivity
- AI compatibility

The project should avoid selecting technologies only because they are popular.

11.18 Architecture Summary

The technical architecture of Construction Project Management & Accounting SaaS will provide a scalable foundation for future growth.

The architecture will support:

- Modern web application development
- SaaS deployment
- Self-hosted deployment
- Mobile accessibility
- AI-assisted development
- Future ERP expansion

A modular architecture combined with a suitable technology stack will allow the platform to grow from MVP level into a complete construction management ecosystem.

End of Chapter 11: Technical Architecture & Technology Stack

Chapter 12: API Design Specification

12.1 Introduction

This chapter defines the API design specification for the Construction Project Management & Accounting SaaS platform.

The API layer will act as the communication bridge between frontend applications, mobile applications, and backend business logic.

A properly designed API structure will ensure:

- Secure communication
 - Easy frontend development
 - Mobile application support
 - Third-party integration capability
 - Future system expansion
-

12.2 API Design Principles

The API design will follow:

- RESTful architecture
 - Version-controlled endpoints
 - Secure authentication
 - Consistent response format
 - Proper validation
 - Clear error handling
-

12.3 API Versioning

All APIs should maintain version numbers.

Example:

```
/api/v1/projects  
/api/v1/employees  
/api/v1/payments
```

Future changes can introduce:

```
/api/v2/projects
```

without breaking existing clients.

12.4 Authentication API

Login

Purpose:

Authenticate users into the system.

Request:

- Email/Phone

- Password

Response:

- Access token
- User information
- Role information
- Tenant information

Logout

Purpose:

Invalidate current user session.

Password Management

Features:

- Forgot password
- Reset password
- Change password

12.5 User Management API

Endpoints:

```
GET    /users
POST   /users
GET    /users/{id}
PUT    /users/{id}
DELETE /users/{id}
```

Operations:

- Create user

- Update user
 - Assign role
 - Activate/deactivate user
-

12.6 Party Management API

Endpoints:

```
GET    /parties
POST   /parties
GET    /parties/{id}
PUT    /parties/{id}
```

Supported party types:

- Client
 - Owner
 - Supplier
 - Subcontractor
-

12.7 Project Management API

Endpoints:

```
GET    /projects
POST   /projects
GET    /projects/{id}
PUT    /projects/{id}
DELETE /projects/{id}
```

Project API should support:

- Project creation
- Status update

- Contract attachment
 - Document attachment
 - Employee assignment
-

12.8 Employee API

Endpoints:

```
GET    /employees
POST   /employees
PUT    /employees/{id}
```

Features:

- Employee creation
 - Assignment
 - Attendance
 - Salary information
-

12.9 Attendance API

Endpoints:

```
POST /attendance
GET  /attendance/report
```

Features:

- Daily attendance
 - Project-wise attendance
 - Working hour tracking
-

12.10 Payroll API

Endpoints:

POST /payroll/generate

GET /payroll/history

Business flow:

Attendance

?

Salary Calculation

?

Payment Record

12.11 Finance API

Endpoints:

GET /transactions

POST /transactions

GET /ledger

Supported transactions:

- Income
 - Expense
 - Payment
 - Adjustment
-

12.12 Document API

Endpoints:

```
POST /documents/upload
```

```
GET /documents
```

```
DELETE /documents/{id}
```

Features:

- Upload files
- Manage access
- Link with projects

12.13 Reporting API

Endpoints:

```
GET /reports/projects
```

```
GET /reports/profit-loss
```

```
GET /reports/expenses
```

Reports:

- Project profitability
- Expense analysis
- Payment history
- Employee cost

12.14 API Security Rules

Every protected API must validate:

1. Authentication
 2. Tenant access
 3. User permission
 4. Business validation
-

12.15 API Response Standard

All APIs should maintain a common response format.

Success:

```
{
  status:true,
  data:{}
}
```

Error:

```
{
  status:false,
  message:"",
  errors:[]
}
```

12.16 API Design Summary

The API architecture will provide a stable communication layer for:

- Web application
- Mobile application
- External integrations

A consistent API structure will make future development faster and reduce maintenance complexity.

End of Chapter 12: API Design Specification

Chapter 13: Frontend Architecture & UI/UX Design

13.1 Introduction

The frontend application of Construction Project Management & Accounting SaaS will provide the user interface for managing construction projects, financial activities, employees, documents, and reports.

The frontend should be designed with focus on:

- Simplicity
- Speed
- Mobile responsiveness
- Easy navigation
- Reusable components
- Business user experience

Since construction owners and site supervisors may use the application from mobile devices, mobile-first design is mandatory.

13.2 Frontend Design Principles

The frontend should follow:

- Component-based architecture
- Responsive design
- Consistent UI patterns
- Reusable components
- Simple workflows
- Minimal user complexity

The system should avoid unnecessary complexity because many users may not have advanced technical knowledge.

13.3 Application Layout Structure

The application layout should contain:

Header

Contains:

- Company logo
- User profile
- Notifications
- Quick actions

Sidebar Navigation

Contains:

- Dashboard
- Projects
- Parties
- Employees
- Finance
- Reports
- Settings

Main Content Area

Displays:

- Forms
 - Tables
 - Reports
 - Dashboards
-

13.4 Dashboard Design

The dashboard should provide an overview of business operations.

Dashboard sections:

Project Summary

Displays:

- Total projects
- Active projects
- Completed projects

Financial Summary

Displays:

- Total income

- Total expense
- Profit/loss

Employee Summary

Displays:

- Total employees
- Pending payments

Alerts

Displays:

- Upcoming payments
 - Project deadlines
 - Notifications
-

13.5 Project Management UI

Project page should include:

Project Overview

Information:

- Project name
- Client
- Contract value
- Status
- Timeline

Project Financial Overview

Displays:

- Income
- Expense
- Profit

Project Activity

Displays:

- Employee assignment
 - Payments
 - Documents
 - Photos
-

13.6 Mobile Responsive Design

The application must support:

- Mobile phones
- Tablets
- Desktop

Mobile design principles:

- Large touch-friendly buttons
- Simple forms
- Optimized tables
- Quick actions

Site supervisors should be able to:

- Upload photos
- Update attendance
- View project information

from mobile devices.

13.7 Form Design Principles

Forms should provide:

- Clear labels
- Proper validation
- Helpful error messages
- Auto-save where necessary

Examples:

Project Creation Form:

- Project name
- Client
- Location
- Contract information

Employee Form:

- Employee details
 - Salary type
 - Assignment
-

13.8 Table Design

Business applications require large data tables.

Tables should support:

- Search
- Filter
- Sorting
- Pagination
- Export

Examples:

- Employee list
 - Transaction list
 - Project list
-

13.9 Report UI Design

Reports should support:

- Charts
- Tables
- Date filters

- Project filters
- Export options

Supported formats:

- PDF
 - Excel
 - CSV
-

13.10 UI Component Strategy

Reusable components should be created for:

- Buttons
- Forms
- Tables
- Modals
- Cards
- Charts
- Notifications

This will reduce development time and improve consistency.

13.11 Frontend Performance Requirements

The frontend should optimize:

- Loading speed
- API requests
- Image loading
- Large data rendering

Techniques:

- Lazy loading
 - Pagination
 - Caching
 - Optimized assets
-

13.12 Frontend Summary

The frontend architecture will provide a simple, responsive, and scalable user experience.

The design goal is to create a professional construction management application usable by both office staff and field workers.

End of Chapter 13: Frontend Architecture & UI/UX Design

Chapter 14: Testing Strategy

14.1 Introduction

Testing is a critical part of Construction Project Management & Accounting SaaS development.

The testing strategy ensures that:

- Business rules work correctly
 - Financial calculations are accurate
 - User permissions are secure
 - System performance remains stable
-

14.2 Testing Approach

The system will follow multiple testing levels:

- Unit Testing
 - Integration Testing
 - API Testing
 - End-to-End Testing
 - Security Testing
 - Performance Testing
-

14.3 Unit Testing

Unit testing will verify individual components and business logic.

Examples:

- Salary calculation
 - Profit calculation
 - Permission validation
 - Expense calculation
-

14.4 Integration Testing

Integration testing verifies communication between modules.

Examples:

Employee Payment:

Employee Module

?

Finance Module

?

Report Module

14.5 API Testing

API testing should verify:

- Request validation
 - Authentication
 - Authorization
 - Response format
 - Error handling
-

14.6 End-to-End Testing

E2E testing should simulate real user workflows.

Example:

Create Company

?

Create Project

?

Add Employee

?

Record Attendance

?

Generate Salary

?

Receive Client Payment

?

View Profit Report

14.7 Security Testing

Security testing should verify:

- Tenant isolation
 - Permission control
 - Authentication security
 - File access control
-

14.8 Financial Testing

Financial calculations require special testing.

Examples:

- Income calculation

- Expense calculation
 - Employee wages
 - Project profit
-

14.9 Testing Automation

Automated testing should be introduced for:

- Critical workflows
 - Financial logic
 - API validation
-

14.10 Testing Summary

A proper testing strategy will ensure reliability, security, and accuracy of the platform.

End of Chapter 14: Testing Strategy

Chapter 15: Reporting & Analytics Design

15.1 Introduction

Reporting is one of the most important parts of the Construction Project Management & Accounting SaaS platform.

The reporting system will help construction owners understand:

- Project performance
 - Financial condition
 - Employee cost
 - Business profitability
-

15.2 Report Design Principles

Reports should be:

- Accurate
 - Easy to understand
 - Filterable
 - Exportable
 - Project-focused
-

15.3 Project Reports

Project reports include:

- Project summary
 - Project income
 - Project expenses
 - Project profit/loss
 - Project timeline
-

15.4 Financial Reports

Financial reports include:

- Income report
 - Expense report
 - Cash flow
 - Ledger report
 - Profit and loss statement
-

15.5 Employee Reports

Employee reports include:

- Attendance report

- Salary report
 - Project assignment report
 - Payment history
-

15.6 Party Reports

Party reports include:

- Client payment history
 - Supplier ledger
 - Subcontractor balance
-

15.7 Analytics Dashboard

Future analytics may include:

- Project profitability comparison
 - Cost trends
 - Employee cost analysis
 - Expense prediction
-

15.8 Reporting Summary

The reporting system will provide business intelligence for better construction management decisions.

End of Chapter 15: Reporting & Analytics Design

Chapter 16: Notification & Automation Design

16.1 Introduction

The notification system will provide automatic alerts based on important business events.

The purpose is to reduce manual tracking and improve operational efficiency.

16.2 Notification Types

Supported notifications:

- Payment reminders
 - Salary reminders
 - Project deadline alerts
 - Equipment return reminders
 - System alerts
-

16.3 Notification Channels

Future supported channels:

- In-app notification
 - Email
 - SMS
 - Push notification
 - WhatsApp
-

16.4 Automation Rules

Examples:

Payment Due Date

?

Generate Reminder

Salary Date

?

Notify Accountant

Project Deadline Near

?

16.5 Notification Management

Users should be able to configure:

- Notification preference
 - Alert type
 - Frequency
-

16.6 Summary

The notification system will improve productivity and reduce missed business activities.

End of Chapter 16: Notification & Automation Design

Chapter 17: SaaS Subscription & Licensing Design

17.1 Introduction

This chapter defines the SaaS licensing and subscription structure.

The system should support flexible business models including:

- Cloud SaaS
 - Self-hosted licensing
 - Enterprise deployment
-

17.2 SaaS Tenant Model

Each company will operate as a separate tenant.

Tenant controls:

- Users
 - Projects
 - Data
 - Settings
-

17.3 Subscription Management

Future subscription features:

- Monthly plans
 - Yearly plans
 - User limits
 - Storage limits
 - Feature packages
-

17.4 License Management

For self-hosted customers:

The system may support:

- License key
 - Activation system
 - Customer verification
 - Renewal management
-

17.5 Future Expansion

Subscription features can be added later without changing core architecture.

End of Chapter 17: SaaS Subscription & Licensing Design

Chapter 18: Maintenance, Monitoring & Future Expansion

18.1 Introduction

This chapter defines long-term maintenance, monitoring, and expansion strategies.

The objective is to keep the platform stable while continuously improving features.

18.2 System Maintenance

Maintenance activities:

- Bug fixing
 - Security updates
 - Performance improvement
 - Dependency updates
-

18.3 Monitoring

The system should monitor:

- Server performance
 - Database performance
 - Application errors
 - User activities
-

18.4 Backup Management

Backup strategy:

- Database backup
- File backup
- Configuration backup

18.5 Future Module Expansion

Possible future modules:

- Inventory management
- Material management
- Procurement
- BOQ management
- Client portal
- Quality management
- Safety management
- AI cost prediction

18.6 Final Architecture Vision

Construction Project Management & Accounting SaaS will evolve into a complete Construction ERP platform.

The architecture will allow:

- Easy module addition
- SaaS growth
- Enterprise adoption
- AI-powered development

End of Chapter 18: Maintenance, Monitoring & Future Expansion

Chapter 19: AI Prompt System & Development Rules

19.1 Introduction

Construction Project Management & Accounting SaaS will be developed using AI-assisted development tools to increase development speed, reduce repetitive tasks, and maintain consistent code quality.

Since AI coding assistants generate output based on provided context, instructions, and existing code

structure, a well-defined AI prompt system is required.

This chapter defines the standard AI development workflow, prompt structure, context management strategy, and coding rules that must be followed throughout the project lifecycle.

The objective is to ensure that AI-generated code remains:

- Architecturally correct
 - Secure
 - Maintainable
 - Scalable
 - Consistent with project requirements
-

19.2 AI Development Philosophy

AI will be used as a development assistant, not as an independent decision maker.

Human decisions are required for:

- Architecture design
- Business rules
- Module planning
- Security decisions
- Database strategy

AI responsibilities:

- Generate implementation code
 - Suggest improvements
 - Create boilerplate
 - Generate tests
 - Assist debugging
 - Improve documentation
-

19.3 Master AI Context System

A master context document should be maintained for AI tools.

The master context will contain:

Project Identity

Information:

- Project name
- Business purpose
- Target users
- Main objectives

Technical Context

Information:

- Technology stack
- Architecture style
- Database choice
- Deployment strategy

Development Rules

Information:

- Coding standards
- Security requirements
- Database rules
- Module structure

Business Context

Information:

- Construction workflow
 - Financial rules
 - Employee management rules
 - Project management rules
-

19.4 AI Context Folder Structure

The project should maintain a dedicated AI context directory.

Recommended structure:

```
/ai-context  
  
01-project-overview.md  
  
02-architecture-rules.md  
  
03-database-rules.md  
  
04-security-rules.md  
  
05-coding-standards.md  
  
06-api-rules.md  
  
07-ui-rules.md  
  
08-module-context/  
    project.md  
    employee.md  
    payroll.md  
    finance.md  
    reporting.md
```

19.5 Prompt Design Standard

Every AI prompt should follow a standard structure.

A complete prompt should contain:

1. Role Definition

Define AI responsibility.

Example:

"You are a senior full-stack software architect."

2. Project Context

Explain the current project.

Example:

"You are working on Construction Project Management & Accounting SaaS."

3. Existing Architecture

Mention:

- Frontend technology
 - Backend technology
 - Database
 - Folder structure
-

4. Feature Requirement

Clearly explain what needs to be developed.

5. Business Rules

Provide related business logic.

Example:

"Employee salary calculation must support daily wage and monthly salary."

6. Technical Requirements

Mention:

- Required files
 - Coding style
 - Security requirements
 - Testing requirements
-

7. Expected Output

Clearly define what AI should generate.

Example:

Generate:

- Database schema
 - API design
 - Backend implementation
 - Frontend components
 - Tests
-

19.6 Master Development Prompt Template

Standard template:

```
You are a senior software architect and developer.
```

```
Project:
```

```
Construction Project Management & Accounting SaaS.
```

```
Your responsibility:
```

```
Develop production-ready code following existing architecture.
```

```
Before generating code:
```

- ```
1. Understand existing structure.
2. Follow project rules.
3. Do not introduce unnecessary complexity.
4. Maintain security requirements.
5. Maintain database consistency.
```

```
Feature:
```

```
[Feature Name]
```

```
Requirements:
```

```
[Requirement Details]
```

```
Business Rules:
```

```
[Business Rules]
```

```
Technical Requirements:
```

```
[Technical Instructions]
```

```
Expected Output:
```

```
[Required Implementation]
```

---

## 19.7 Module Development Prompt Strategy

---

Each module should have its own AI context.

Example:

Project Module Context:

Project Module

Purpose:

Manage construction projects.

Related Entities:

- Project
- Party
- Contract
- Document

Business Rules:

- Project belongs to tenant.
- Project must have client.
- Contract is optional.

AI should receive only the required module context.

---

## 19.8 Token Optimization Strategy

---

AI tools have limited context capacity.

To reduce unnecessary token usage:

### Do Not Provide:

---

- Entire documentation
- Unrelated modules
- Old discussions

### Provide:

---

- Current module requirement
- Related business rules

- Database structure
- Coding rules

Example:

When developing Payroll Module:

Required:

- Employee rules
- Payroll rules
- Finance transaction rules

Not required:

- Machinery module
- Equipment rental module

---

## 19.9 AI Coding Rules

---

AI-generated code must follow:

### Clean Architecture

---

Code should maintain:

- Proper separation
- Clear responsibility
- Reusable components

---

### No Duplicate Logic

---

Common functionality must be reusable.

Example:

Payment calculation logic should exist in one place only.

---

## Security First

---

AI must always consider:

- Authentication
  - Authorization
  - Tenant isolation
  - Input validation
- 

## No Hardcoded Values

---

Avoid:

- Passwords
- API keys
- Configuration values

Use:

- Environment variables
  - Configuration files
- 

## 19.10 Database Generation Rules

---

When AI generates database code:

Must consider:

- Relationships
- Foreign keys
- Indexing
- Tenant isolation
- Audit fields

Common fields:

```
id
tenant_id
created_at
```

```
updated_at
```

```
created_by
```

```
updated_by
```

---

## 19.11 Backend Development Rules

---

Every backend module should contain:

```
Module
```

```
??? Controller
```

```
??? Service
```

```
??? Repository
```

```
??? DTO
```

```
??? Entity
```

```
??? Validation
```

```
??? Tests
```

AI should not mix business logic inside controllers.

---

## 19.12 Frontend Development Rules

---

Frontend code should follow:

- Component reuse
- Responsive design
- Mobile-first approach
- Proper state management
- Form validation

AI should avoid creating large single components.

---

## 19.13 AI Testing Prompt Strategy

---

Every important feature should include testing.

Example:

Generate tests for Project Payment Workflow.

Include:

- Successful payment
- Invalid payment
- Permission failure
- Tenant isolation
- Calculation validation

---

## 19.14 AI Code Review Process

---

Before accepting AI-generated code:

Run AI review.

Review areas:

### Architecture

---

- Correct module placement
- Proper dependency management

### Security

---

- Permission validation
- Data protection

### Database

---

- Relationship correctness
- Performance

## Business Logic

---

- Requirement compliance

## Code Quality

---

- Maintainability
  - Readability
- 

## 19.15 AI Debugging Workflow

---

When fixing bugs:

Do not ask AI only:

"Fix this error."

Instead provide:

1. Problem description
2. Expected behavior
3. Actual behavior
4. Error message
5. Related code

Example:

```
Problem:
Employee salary calculation is incorrect.

Expected:
Daily wage × working days.

Actual:
Wrong amount generated.

Analyze root cause and provide solution.
```

---

## 19.16 AI Feature Development Workflow

---

Every feature should follow:

Requirement

?

Business Rule

?

Database Design

?

API Design

?

Backend Development

?

Frontend Development

?

Testing

?

Code Review

AI should not skip architectural planning.

---

## 19.17 AI Documentation Update Rules

---

After completing a module:

AI should update:

- API documentation
- Database documentation
- Module documentation
- Development notes

Documentation should always match current implementation.

---

## 19.18 Future AI Development Improvements

---

Future improvements may include:

- Automated code generation templates
  - Reusable prompt library
  - AI-powered test generation
  - AI documentation assistant
  - Automated code review
- 

## 19.19 Final AI Development Guidelines Summary

---

The AI development system will ensure that AI-generated code follows a predictable and professional development process.

The key principles are:

- Provide clear context
- Use structured prompts
- Develop module by module
- Protect architecture
- Maintain security
- Review all generated code

A strong AI prompt system will reduce development cost, save tokens, improve output quality, and allow faster development of Construction Project Management & Accounting SaaS.

---

**End of Chapter 19: AI Prompt System & Development Rules**